SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

FACULTY OF COMPUTING.

WEB SECURITY- IE2062

IT22121592 – HERATH H.M.C.H.K

Y2S2

Contents

Subdomain Enumeration: 3
Vulnerability title. 3
Vulnerability description. 4
Affected components. 5
Impact assessment. 7
Steps to reproduce. 9

My second subdomain was ‘login.bigcommerce.com’.

Subdomain Enumeration:

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ ping login.bigcommerce.com
PING shared-lb-lb1.gcp.bigcommerce.net (35.227.219.245) 56(84) bytes of data.
64 bytes from 245.219.227.35.bc.googleusercontent.com (35.227.219.245): icmp_seq=1 ttl=115 time=139 ms
64 bytes from 245.219.227.35.bc.googleusercontent.com (35.227.219.245): icmp_seq=2 ttl=115 time=74.6 ms
64 bytes from 245.219.227.35.bc.googleusercontent.com (35.227.219.245): icmp_seq=3 ttl=115 time=813 ms
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ dig login.bigcommerce.com @8.8.8.8

; <<>> DiG 9.19.21-1-Debian <<>> login.bigcommerce.com @8.8.8.8
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4005
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;login.bigcommerce.com.         IN      A

;; ANSWER SECTION:
login.bigcommerce.com.  403     IN      CNAME   shared-lb-lb1.gcp.bigcommerce.net.
shared-lb-lb1.gcp.bigcommerce.net. 28 IN A      35.227.219.245

;; Query time: 96 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Thu Oct 17 20:59:03 +0530 2024
;; MSG SIZE  rcvd: 113
```

Vulnerability title.

Boolean-based Blind SQL injection.

```
[21:28:21] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[21:28:44] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
```

Vulnerability description.

Trying to crack a password for an account feels like solving a mystery puzzle where the system doesn't just say "yes" or "no" when you make a guess. For responses, like that, whether it’s a regular page or an error message, you have to pay attention and piece together the password bit, by bit.

This is how it operates:

Locating a Vulnerable Point: Initially pinpoint an area, on the website, such as a login section or search field. Where there exists a vulnerability, in the system's defenses; this serves as an entry point to inject code discreetly.

Testing the code: Next is sending a code snippet to see if a part of the system reacts with either a "true" or "false." Although immediate data isn't provided at this point in time, however you can gauge your progress based on how the system responds.

Determining the accuracy of your guess can be done by observing the website's reaction, such, as loading a page or displaying an error page.

Repeating the Strategy: You continue making educated guesses gradually uncovering bits of information step by step; similar, to discovering the letter of a password first and then proceeding to decipher the subsequent ones sequentially. Despite receiving no confirmation from the system itself; you can deduce the solution by observing patterns, in its responses.

Here is a straightforward demonstration;

Imagine you're attempting to figure out a password in a system and the system could have a concealed prompt, to this:

```
SELECT * FROM users WHERE username = 'admin' AND password = 'password123';
```

Retrieve all information of users whose username is 'admin’. The password is 'password123’, from the database.

You may not have the password, on hand. You could insert a code snippet to verify if the initial letter of the password is 'A'. As an illustration of this idea, a typical code snippet could appear like the following;

```
' OR (SELECT SUBSTRING(password, 1, 1)='a')—
```

If the system operates as usual and gives you a result. Like the first letter being 'A' you can proceed with confidence that you're, on the track. However, if there’s an error or a different outcome occurs after your input. Then it's safe to assume that 'A' is not the letter; in cases, you can consider trying out other options like 'B' and 'C' and continue in a similar manner. By repeating this process persistently over time. Testing possibilities one, by one. You will eventually be able to deduce the complete password step by step.

Why the term "blind" is used:

It's known as "blind" since you don't immediately observe the real data upfront. You're navigating in the shadows. Can deduce information by observing how the website responds to your assumptions.

It's a bit similar, to playing a round of 20 questions. You ask simple Yes/ No queries to unravel the mystery without getting the answer. When websites do not provide information directly this method may be slow yet efficient.

Affected components.

Boolean-based Blind SQL Injection has the potential to impact aspects of a web application, including its underlying infrastructure components that are susceptible, to or affected by such attacks;

1 -SQL databases: SQL injection attacks that specifically target the database's SQL queries being run by it. This vulnerability enables attackers to access and manipulate data like user credentials and financial information such, as credit card details.

Sensitive information such, as details or financial records stored in the database may be, at risk of exposure or unauthorized access.

2- Website application:

User Input Fields such, as login forms and search bars are targets, for injecting SQL code.

When queries are manipulated inappropriately in the applications logic system can. Exhibit behavior that may result in security vulnerabilities or unauthorized access escalations.

Mistakes and Feedback: The program might accidentally disclose details, in its error notifications. Exhibit changes, in its behavior, based on whether the SQL conditions are valid or invalid which could be triggered by an attacker.

3- Server-Side Scripting: Sever-side scripting is an aspect of web development that involves writing code that runs on the server to generate content, for websites.

The server-side code of a web application (like PHP, ASP.NET, or Java) known as the back-end code for handling SQL queries can become vulnerable, to SQL injection attacks if it fails to sanitize inputs or utilize prepared statements.

Authentication Methods. Malicious actors may be able to circumvent the login authentication process if the web applications query used to verify login credentials is susceptible, to SQL injection attacks.

4- Web Server: Improper setup of a web server could potentially reveal details to individuals and simplify their task of exploiting SQL injection weaknesses; for instance. Thorough error reporting could assist attackers in creating more accurate injection commands.

Server Records Alert: Sometimes the logs stored on the server might mistakenly reveal data that could aid hackers in enhancing their SQL injection strategies.

5- Data Transmission Concern: SQL injection attacks can impact the reliability of data moving between the software and the database system. For example, a malicious actor could insert queries that alter how data appears or is stored which could lead to the disclosure of information.

6- API Security Risks: When an application makes its APIs accessible to interact with the database directly and they don't properly handle user input validation or sanitization procedures in place to avoid security risks, like SQL injection attacks.

7- Third-party Extensions: In systems such, as WordPress or Joomla and other content management platforms (CMS) third-party add-ons that fail to filter user inputs could pose a security risk by leaving the system vulnerable, to SQL injection attacks.

"Storing Data, for Quick Retrieval Systems;"

8- Web Caches can be vulnerable, to manipulation, by attackers who could misuse the way web pages store information retrieved from SQL queries. This could lead to caching systems displaying content depending upon injected criteria resulting in a potential risk of exposing confidential data to unauthorized individuals.

9- Logs that are not properly configured in logging systems could fail to detect SQL injection attempts or potentially reveal information, to attackers, such, as database error messages.

By taking advantage of weaknesses in any of these parts of the system infrastructure an attacker could alter the way SQL queries are run to obtain entry, to the system and potentially tamper with or erase data which could ultimately jeopardize the security of the entire system.

Impact assessment.

When a web app is susceptible, to Boolean-based Blind SQL Injection attacks it can result in ramifications, across sections of the system. Let's delve into the effects:

One potential issue is a data breach:

1- Access to Sensitive Data: Cybercriminals may obtain data like usernames and passwords or sensitive financial and personal information, through covert means without direct visibility of the data itself by employing a series of yes/no inquiries to gradually uncover this confidential information.
2- Disclosure of Information: Sharing internal data without the permission of the rightful owner could lead to breaches of privacy regulations, like GDPR or HIPAA.

Tampering with Data Security involves changes, to information stored in the database such, as modifying user roles or financial records which can lead to compromised accuracy and integrity of the data.

Loss of Data Integrity Issue: When crucial application data is altered it may lead to database corruption or undermine confidence, in the reliability of the system.

Unauthorized Entry Alert!

Cyber intruders have the ability to create SQL injections in order to circumvent security measures, for access control systems. In a scenario where they alter a query to consistently return a result this could enable them to gain privileges without possessing the password.

By exploiting SQL queries in a way malicious actors can boost their access privileges to administrator levels, within the system. This enables them to carry out alterations or deactivate security measures.

SQL injection can lead to a **Denial of Service (DoS) attack** by flooding the database with queries that could cause the database server to crash or render the web application inaccessible, to users.

Service Disruption Alert:

Continual attempts to breach security or successful hacking could result in service interruptions or downtime that affect user satisfaction and business functionality.

Failure to follow rules:

When data breaches occur due, to SQL injection attacks it could lead to violations of data protection laws such as GDPR or HIPAA. Result in fines as well as legal consequences including mandatory notifications, to those affected by the breach.

Exposing a breach caused by a vulnerability such, as SQL injection can significantly harm an organization's credibility and trustworthiness in the eyes of customers and partners who may doubt the effectiveness and reliability of its security measures.

In case financial information or transactions are altered intentionally it may lead to direct thefts or frauds by manipulating the finances.

The expense of remediating issues such, as fixing vulnerabilities and investigating breaches can be significant due, to the costs associated with labor hours lost and the resources needed for restoration efforts.

The decrease, in income due to the disruption caused by the cyber attack and the erosion of customer confidence could lead to a drop, in revenue and possible loss of clientele.

Engaging in a series of attacks:

Exploiting SQL injection vulnerabilities can act as a gateway, to launching attacks. If a hacker manages to get in they can then move on to parts of the network. Take advantage of other security gaps.

After infiltrating a system successfully ،attackers could navigate through the network from, within to reach systems and data.

If the application contains information, in its database like property or trade secrets could be at risk of theft, by attackers.

Exploiting vulnerabilities to install software:

Compromising Application Logic: When attackers gain control through SQL injection attacks and inject code into the application (such, as installing backdoors or delivering malware) it can result in compromises to the system.

Loss of customer trust can be an issue:

For businesses when their services are compromised and sensitive personal information is exposed.

Damage to Partner Relationships: If companies are seen as having security vulnerabilities that could impact their partner's systems it may lead to distancing, between partners and vendors.

Steps to reproduce.

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ curl -v https://login.bigcommerce.com
* Host login.bigcommerce.com:443 was resolved.
* IPv6: (none)
* IPv4: 35.227.219.245
*   Trying 35.227.219.245:443...
* Connected to login.bigcommerce.com (35.227.219.245) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519 / RSASSA-PSS
* ALPN: server accepted http/1.1
* Server certificate:
*  subject: C=US; ST=Texas; L=Austin; O=BigCommerce, Inc.; CN=*.bigcommerce.com
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ curl -v https://login.bigcommerce.com
* Host login.bigcommerce.com:443 was resolved.
* IPv6: (none)
* IPv4: 35.227.219.245
*   Trying 35.227.219.245:443...
* Connected to login.bigcommerce.com (35.227.219.245) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
*  CAfile: /etc/ssl/certs/ca-certificates.crt
*  CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519 / RSASSA-PSS
* ALPN: server accepted http/1.1
* Server certificate:
*  subject: C=US; ST=Texas; L=Austin; O=BigCommerce, Inc.; CN=*.bigcommerce.com
```

```
*  Certificate level 0: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
*  Certificate level 1: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
*  Certificate level 2: Public key type RSA (2048/112 Bits/secBits), signed using sha256WithRSAEncryption
* using HTTP/1.x
> GET / HTTP/1.1
> Host: login.bigcommerce.com
> User-Agent: curl/8.5.0
> Accept: */*
>
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* old SSL session ID is stale, removing
< HTTP/1.1 401 Unauthorized
< Content-Type: application/json; charset=utf-8
< Transfer-Encoding: chunked
< Connection: keep-alive
< referrer-policy: strict-origin-when-cross-origin
< cache-control: no-store
< pragma: no-cache
< vary: Accept
< content-language: en
< x-content-type-options: nosniff
< content-security-policy: default-src 'self' https:; frame-ancestors 'self' https://*.mybigcommerce.com; im
'self' 'unsafe-inline' https:
< x-request-id: 73c73565020ecb821054d4f90ab040e4
< x-envoy-upstream-service-time: 9
< Strict-Transport-Security: max-age=31536000; includeSubDomains
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ curl -I https://login.bigcommerce.com
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8
Connection: keep-alive
referrer-policy: strict-origin-when-cross-origin
cache-control: no-store
pragma: no-cache
vary: Accept
content-language: en
x-content-type-options: nosniff
content-security-policy: default-src 'self' https:; frame-ancestors 'self' https://*.mybigcommerc
elf' 'unsafe-inline' https:
x-request-id: b133f3377d681b8586acb50a793219ae
x-envoy-upstream-service-time: 3
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

Sqlmap:

```
[21:28:21] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[21:28:44] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
```

```
┌──(jeon㉿demo)-[~/Sublist3r]
└─$ nikto -h https://login.bigcommerce.com
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          35.227.219.245
+ Target Hostname:    login.bigcommerce.com
+ Target Port:        443
---------------------------------------------------------------------------
+ SSL Info:        Subject:  /C=US/ST=Texas/L=Austin/O=BigCommerce, Inc./CN=*.bigcommerce.com
                   Ciphers:  TLS_AES_256_GCM_SHA384
                   Issuer:   /C=US/O=DigiCert Inc/CN=DigiCert Global G2 TLS RSA SHA256 2020 CA1
+ Start Time:         2024-10-17 21:42:16 (GMT5.5)
---------------------------------------------------------------------------
+ Server: No banner retrieved
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.or
+ /: Uncommon header 'x-envoy-upstream-service-time' found, with contents: 8.
+ /: Uncommon header 'x-request-id' found, with contents: 2c136dfd864334450f892781f0da8666.
+ Root page / redirects to: https://login.bigcommerce.com/login
+ /DDxXFFbc.nl: The X-Content-Type-Options header is not set. This could allow the user agent to re
ties/missing-content-type-header/
```

1- Proposed mitigation or fix.

1- Ensure that user inputs are verified and sanitized before proceeding:

Running a restaurant requires control, over who enters the kitchen to ensure safety and quality standards are upheld at all times – like how you need to verify the authenticity and safety of user inputs by ensuring they are, in the right format and don't contain any harmful elements before accepting them into your system as this simple step helps protect against potential security breaches by malicious actors.

2- Utilize prepared statements for creating your SQL queries!

Imagine it as using a form, with blanks to fill in information in crafting a SQL query that directly inserts user details (which could pose risks). You develop a template", for your SQL query where user inputs fit into designated spots without changing the query format of what users input – just like securing doors to prevent break-ins!

Here is an example;

```
SELECT * FROM users WHERE username =? AND password =?
```

The placeholders (?) are used to make sure that the inputs are not considered as part of the SQL command.

3- Limit access, to the database (Avoid granting access) :

At a workplace or organization, not all individuals require permission to view papers. Similarly, the database logins utilized by your software must be restricted to performing tasks. No extra privileges should be granted. For example, if your program is solely designed for data retrieval purposes refrain from authorizing it to erase any information. This practice ensures that in the event of a security breach, the intruder's scope of access remains restricted.

4- Make sure not to provide error messages that could hint at the issue at hand keeping them general to maintain security and privacy.

When an issue arises in the operation of your system and you feel the urge to display error messages, for troubleshooting purposes; it's important to consider that malicious individuals could leverage this data to pinpoint vulnerabilities in your system structure instead show a

message such as "An error has occurred; please attempt again at a later time" and maintain comprehensive logs discreetly accessible only, to developers.

5- Utilize a Web Application Firewall (referred to as WAF) which acts as a security guard, for your platform.

Think of a Web Application Firewall as a shield, for your web application—it works like a security guard that keeps an eye on traffic and stops any shady activities like SQL injection attempts before they can harm your application's integrity! While it's a defense measure to have in place, against cyber threats and attacks it's essential to remember that it's not fail-safe, consider it just one piece of the security puzzle you need to have in place to protect your online assets effectively.

6- Regularly conduct security assessments to identify and address any vulnerabilities, in the system.

Just as you make sure to have your car serviced regularly for upkeep and maintenance purposes; it's important to conduct checkups, on your web application! Performing security audits and penetration tests is crucial in pinpointing vulnerabilities such, as SQL injection points before they are exploited by attackers.

7- Utilize security tools and frameworks for protection, against cyber threats.

To start everything from one consider using established frameworks and libraries that come with built-in security features. For instance, numerous contemporary web development frameworks take care of input validation. SQL queries automatically decrease the likelihood of SQL injection without requiring you to do all the difficult work.

8- Educate developers, on the importance of security measures, in software development.

Developers often prioritize functionality, over security considerations at times. Security training sessions serve as a way to keep developers informed about vulnerabilities such, as SQL injection and ways to prevent them. Imagine it as a reminder for your team to lock the doors and windows securely!

9- Remember to ensure that all your software and systems are kept up, to date.

It's important to update your web server and software just as you would update your phone's operating system, for security reasons. This ensures that any vulnerabilities are patched up promptly and helps prevent exploitation, by attackers.

10- Make sure you have a plan, in case things don't go as expected.

Despite having defenses, in place at all times situations can still go awry. It is crucial to have a strategy for how to react in the event of an attack. This plan may involve procedures for examining the breach resolving the problem and notifying those impacted. It's akin to having a fire evacuation strategy—you hope you never have to use it. If the need arises you'll be grateful that it's, in place.

By following these guidelines you're essentially safeguarding your web app making it more challenging, for intruders to infiltrate. Similar to securing a home it's all about having levels of security, from locks on the doors to a security system that notifies you in case of any issues.